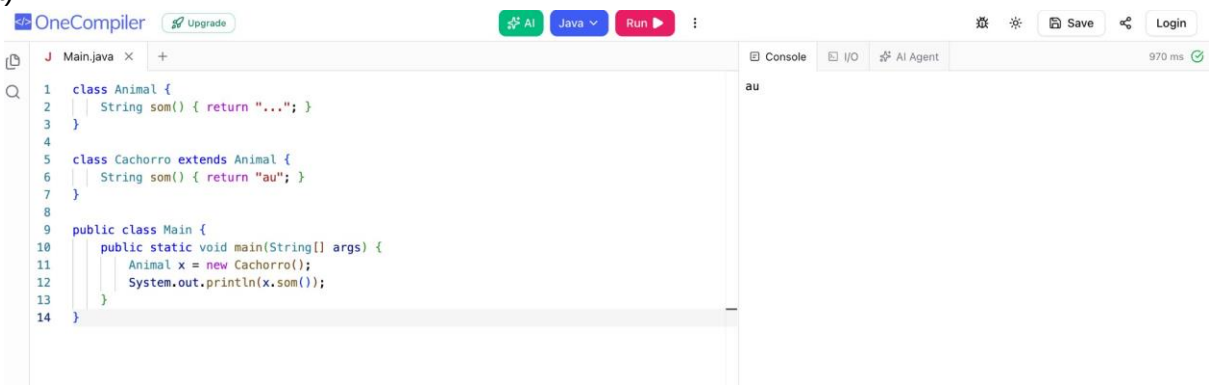
Atividade aula-009 - Paradigmas e Linguagens

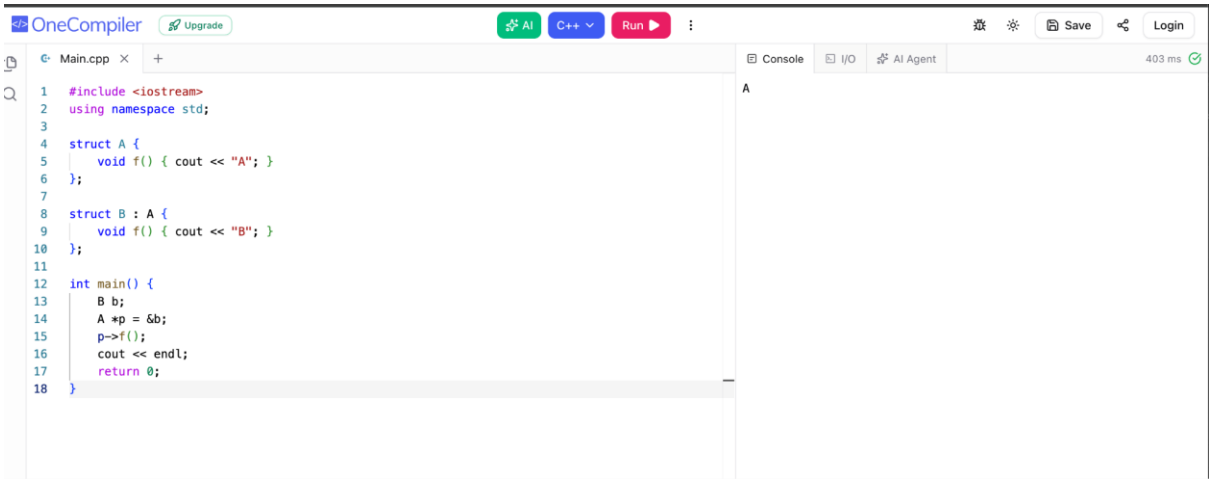
Sophia Machado Silva - RA: 24087451-2

1. a)



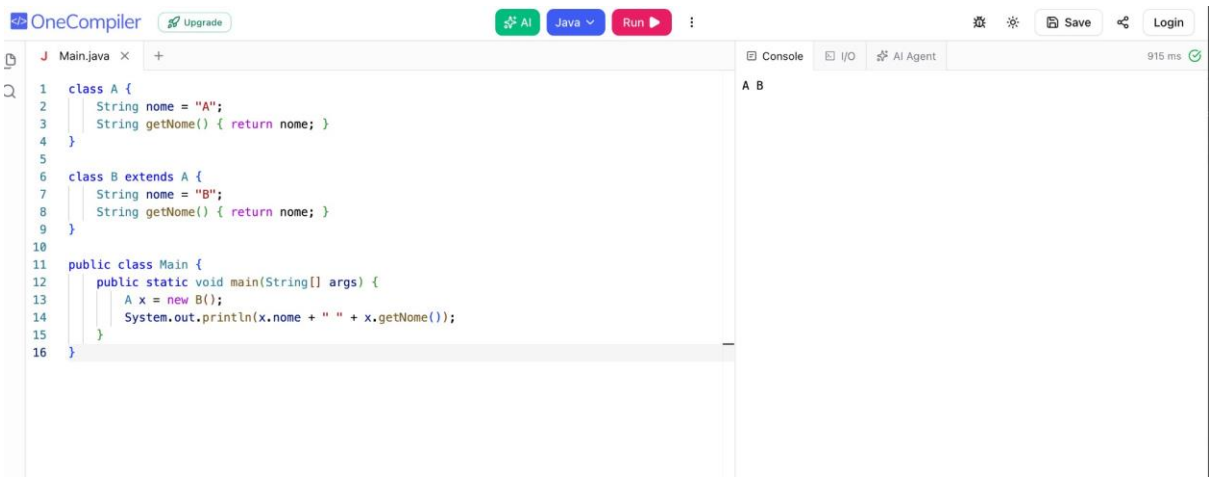
b) Execução. Mesmo com a variável declarada como Animal, o objeto criado é um Cachorro. A JVM olha para o objeto real na hora da chamada e executa o som() dele. É vinculação dinâmica.

2. a)



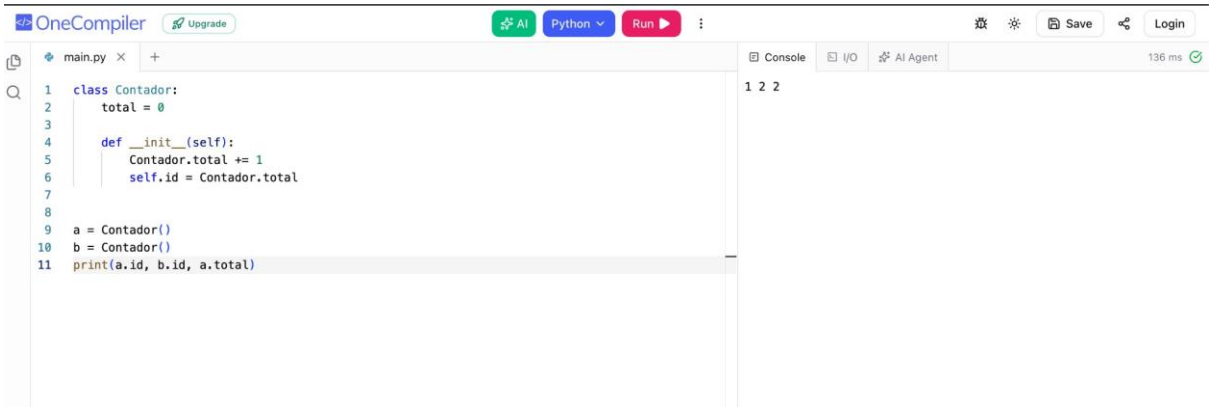
b) Compilação. Como f não foi declarada virtual, o compilador resolve a chamada só pelo tipo do ponteiro (A*) e já fixa A::f. Se A tivesse virtual void f(), a saída seria B.

3. a)



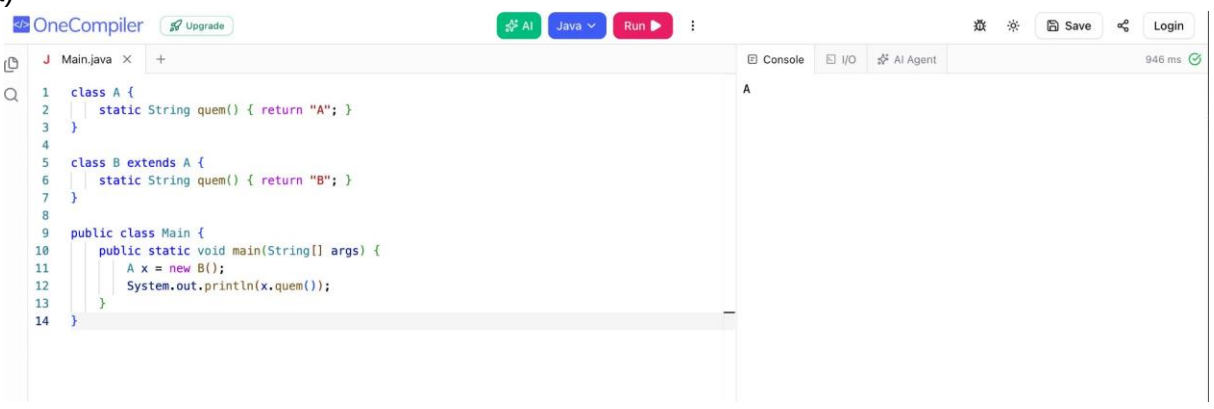
b) Compilação para o campo, execução para o método. O x.nome usa o tipo declarado da variável (A), porque campos não são polimórficos e o nome de B apenas esconde o de A. Já o x.getNome() é despachado dinamicamente para a versão de B, que devolve o nome de B. Por isso é boa prática manter os campos private.

4. a)



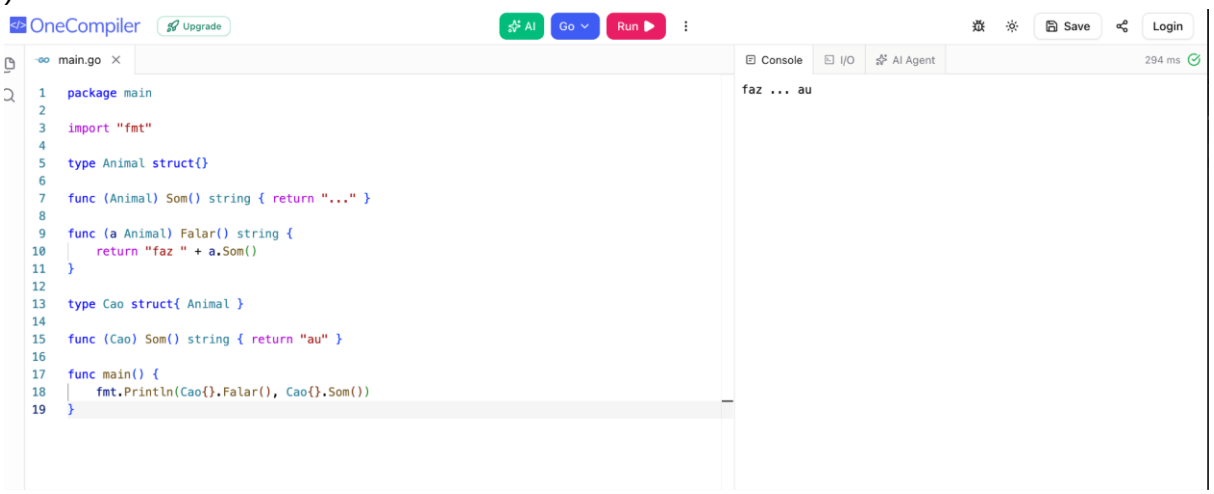
b) Execução. Python resolve os nomes ao rodar. O total é uma variável de classe, com uma única cópia compartilhada, e chega a 2 depois dos dois objetos. O id é de instância, cada objeto guarda o seu (1 e 2). O a.total não existe em a, então o Python busca na classe e encontra 2.

5. a)



b) Compilação. Método static não é sobrescrito, é escondido. A chamada é resolvida pelo tipo declarado da variável (A), então roda o quem() de A. O compilador ou a IDE até avisa que o certo seria chamar A.quem().

6. a)



b) Compilação. Embedding em Go não é herança. Dentro de Animal.Falar, a.Som() sempre chama o Som de Animal, porque não há despacho dinâmico entre o struct embutido e o que o contém. Cao{}.Som() chama o Som do próprio Cao. Para ter polimorfismo em Go é preciso usar uma interface.